1.
O(log k) when k = max(x,y)/min(x,y)
ceil(log(max(x, y) / min(x, y))

2.
O(nlogn)

3.
<1> f(n, k + 1)
<2> f(n // k, k)
<3> f(n, k + 1)

4.
<1> yield (next(zit1), next(zit2))
<2> StopIteration
<3> break

5.
<1> [sum(x) for x in zip([0]+line, line+[0])]

6.
<1> g(f(st, n-1))
or
<2> f(g(st), n-1)

7.
reduce(lambda x, _: g(x), range(n), s)

8.
```
def countmax(t):
    nd = [0]

    def helper(tr, k):
        if tr.label > k:
            nd[0] += 1

        for b in tr.branches:
            helper(b, max(tr.label, k))

    helper(t, t.label-1)
    return nd[0]
```

9.

```
def balance_depth(expr):
    depth = 0
    number = ''
    for char in expr:
        if char == '(':
            depth += 1
        elif char.isdigit():
            number += char
        elif char == ')':
            if number and int(number) != depth:
                return False
            # reset number after check a complete set of parentheses
            number = ''
            depth -= 1
    return True
```

10.

```
class Matrix:
    def __init__(self, data):
        self.data = data

    def __str__(self):
        return '\n'.join(' '.join(str(x) for x in row) for row in self.data)

    def transpose(self):
        return Matrix(tuple(zip(*self.data)))

    def flip_rows(self):
        return Matrix(self.data[::-1])

    def flip_cols(self):
        return Matrix(tuple(row[::-1] for row in self.data))

    def rotate(self):
        rotated = self
        while True:
            rotated = rotated.transpose().flip_cols()
            yield rotated
```

11.

```
def link_comprehension(lnk, mp, filt):
    if lnk is Link.empty:
        return Link.empty

    new_next = link_comprehension(lnk.next, mp, filt)
    if filt(lnk.data):
        return Link(mp(lnk.data), new_next)
    else:
        return new_next
```

12.

```
def consecutive_seq(tr):
    def get_long(t):
        max_count = count = 1
        if not t.is_leaf():
            for br in t.branches:
                k, k_max = get_long(br)
                if br.label == t.label + 1:
                    count = max(count, k + 1)
                max_count = max(max_count, count, k_max)
        return count, max_count
    return get_long(tr)[1]
```

13.

```
up = lambda n: helper(up, print, n//10)
down = lambda n: helper(print, down, n//10)
```